

Informe del Proyecto

“Smart Astronaut”

Wendy Yepez Aguayo

Mónica Alejandra Cortés

Jhoimar Silva Torres

Jean Giraldo Osorio

Inteligencia Artificial

Oscar Bedoya Leiva

El presente proyecto desarrolla e implementa diferentes algoritmos de búsqueda aplicados a un entorno simulado en el cual un astronauta explora un planeta desconocido representado en una matriz o “mundo”. El objetivo principal es que el astronauta recolecta tres muestras científicas dispersas por el terreno, evitando obstáculos y utilizando una nave espacial que le permite desplazarse temporalmente con menor costo. Cada celda del mapa representa un tipo de terreno o elemento, y el agente puede moverse en las cuatro direcciones cardinales: Arriba, Abajo, Izquierda y Derecha. No se permiten movimientos diagonales. El proyecto integra distintos algoritmos de búsqueda, tanto no informados como informados, para analizar y comparar sus resultados en términos de nodos expandidos, profundidad alcanzada y tiempo de cómputo.

La finalidad es aplicar los conceptos teóricos vistos en clase para construir un sistema que demuestre la diferencia entre las estrategias de exploración ciega y las basadas en heurísticas, consolidando así el entendimiento práctico de los algoritmos de búsqueda en Inteligencia Artificial.

Objetivo general

Desarrollar una simulación funcional en Python que permita comparar el desempeño de diferentes algoritmos de búsqueda, aplicados al movimiento de un agente autónomo dentro de un entorno controlado.

Objetivos específicos

- Implementar algoritmos de búsqueda no informada (profundidad evitando ciclos, amplitud, costo uniforme).
- Implementar algoritmos de búsqueda informada (Avara y A*).
- Diseñar una interfaz gráfica que permita visualizar en tiempo real el recorrido del agente y sus decisiones.
- Analizar el desempeño de cada algoritmo con base en nodos expandidos, profundidad y tiempo de ejecución.

- Garantizar que el agente evite ciclos y respete las reglas de movimiento definidas.

Marco teórico

Búsqueda no informada

Estas estrategias no poseen conocimiento adicional del problema más allá de los estados y operadores disponibles.

- Profundidad evitando ciclos (DFS): explora el camino más profundo antes de retroceder, usando una pila. Se implementó con control de nodos visitados para evitar repeticiones y con la posibilidad de retroceder solo por casillas de tipo nave o muestra.
- Amplitud (BFS): explora por niveles, utilizando una cola (FIFO). Es óptima en espacios con costos uniformes.
- Costo uniforme: expande el nodo con menor costo acumulado (usando una cola de prioridad). Considera diferentes tipos de terreno (normal, rocoso, volcánico).

Búsqueda informada

Estas estrategias usan una función heurística $h(n)$ que estima el costo desde el estado actual hasta la meta.

- Avara (Greedy): selecciona el nodo con menor heurística estimada sin considerar el costo acumulado.
- A*: combina el costo real y el estimado mediante $f(n) = g(n) + h(n)$, garantizando soluciones óptimas si la heurística es admisible.

La heurística usada fue la distancia Manhattan, adecuada para movimientos en cuatro direcciones (Arriba, Abajo, Izquierda y Derecha).

Representación del entorno

El mundo se carga desde un archivo de texto (Prueba1.txt) con valores numéricos, donde cada valor representa un tipo de terreno con su costo de movimiento.

Significado	Costo
Terreno normal	1
Obstáculo (no transitable)	
Astronauta (inicio)	
Terreno rocoso	3
Terreno volcánico	5
Nave auxiliar	20 pasos con costo 1/2
Muestra científica (meta parcial)	

El agente puede moverse en las direcciones arriba, abajo, izquierda y derecha, siempre que la casilla destino sea válida (no obstáculo ni fuera de los límites).

Explicación tipos de Búsquedas

1. Búsqueda por Amplitud

- Explora nivel por nivel, expandiendo primero los nodos más cercanos a la raíz.
- Garantiza encontrar la solución más corta en términos de número de pasos.
- No considera costos diferentes por terreno.

2. Búsqueda de Costo Uniforme

- Similar a amplitud, pero elige expandir siempre el nodo con menor costo acumulado.
- Ideal para entornos donde los terrenos tienen diferentes costos de movimiento.
- Requiere una cola de prioridad (por ejemplo, heapq en Python).

3. Búsqueda en Profundidad evitando ciclos

- Se implementó utilizando una pila (LIFO).
- Evita ciclos mediante un conjunto de nodos visitados (visitados).
- Puede devolverse cuando pasa por la nave o una muestra.
- Termina al recolectar las tres muestras científicas.

4. Algoritmo Avara (Greedy Best-First Search)

- Selecciona siempre el nodo que parece más cercano a la meta, según una función heurística $h(n)$.
- No garantiza la mejor solución (óptima), pero suele ser muy rápida.
- En el código `algoritmo_avara.py`:
- Se usa la distancia Manhattan como heurística.
- Implementa una cola de prioridad que elige el menor valor de $h(n)$.
- Al llegar a una muestra, continúa hacia la siguiente, hasta recolectar las tres.
- Registra nodos expandidos, tiempo total, y costo.

5. Algoritmo A*

- Mezcla lo mejor de Costo Uniforme y Avara, combinando:

$$f(n) = g(n) + h(n)$$

donde:

- $g(n)$: costo real acumulado.
- $h(n)$: heurística (distancia estimada a la meta).
- Garantiza encontrar el camino óptimo, siempre que la heurística sea admisible.
- En el código `algoritmo_a_estrella.py`:
 - Utiliza una cola de prioridad.
 - Calcula el costo real considerando los diferentes tipos de terreno (1, 3, 4, etc.).
 - Usa también la distancia Manhattan como heurística $h(n)$.
 - Expande el nodo con menor $f(n)$.
 - Termina cuando recolecta las tres muestras científicas.

Heurística utilizada

La heurística empleada en los algoritmos informados (Avara y A*) es la distancia Manhattan, definida como:

$$h(n) = |x_{meta} - x_{actual}| + |y_{meta} - y_{actual}|$$

Esta mide la cantidad mínima de pasos necesarios para alcanzar la meta, sin considerar obstáculos ni tipos de terreno.

Ventajas:

- Es sencilla de calcular.
- Funciona correctamente en mapas con movimientos solo verticales y horizontales.
- Es admisible (nunca sobreestima el costo real).

Justificación de la admisibilidad de la heurística

Una heurística es admisible si nunca sobreestima el costo real del camino hasta la meta.

La distancia Manhattan cumple esta propiedad porque:

- Calcula el mínimo número posible de movimientos requeridos.
- Ignora obstáculos o terrenos difíciles, por lo que su estimación siempre es igual o menor al costo real.

Por tanto:

$$h(n) \leq h^*(n)$$

donde:

$h^*(n)$ es el costo real óptimo desde n hasta la meta.

Esto garantiza que el algoritmo A* encontrará siempre la solución óptima.

Conclusión general

El proyecto demuestra la aplicación práctica de los algoritmos de búsqueda vistos en clase.

A través de la simulación visual se comprende cómo:

- Los algoritmos no informados exploran sin conocimiento previo (más lentos pero exhaustivos).
- Los algoritmos informados emplean heurísticas para optimizar el proceso y tomar decisiones más inteligentes.

Aprendizaje logrado

Se aplicaron los conceptos de espacio de estados, función de costo, heurísticas, nodos, frontera y ciclos, logrando implementar un entorno completo que integra teoría y práctica de Inteligencia Artificial.